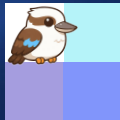




Centre for
Robotics



SLAM Course

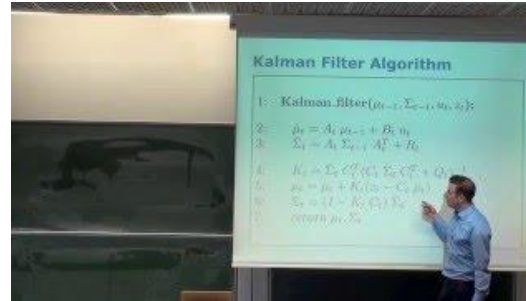
06 - EKF-SLAM

Dr Alejandro Fontán Villacampa
April 2026

Kalman Filter and EKF



Kalman Filter - 5 Minutes with Cyrill
https://youtu.be/o_HW6GnLqvq



Photogrammetry II - 12 - Kalman Filter and EKF (2015/16)
https://youtu.be/BblUVCcT_s



EKF-SLAM (Cyrill Stachniss)
<https://youtu.be/X30sEglws0g>



SLAM Handbook
<https://github.com/SLAM-Handbook-contributors/slam-handbook-public-release>

Probabilistic Robotics
https://www.academia.edu/6819774/PROBABILISTIC_ROBOTICS

<https://github.com/alejandrofontan/SLAM-Course/>



Bayes Filter

- ❖ The Bayes Filter is the general recursive framework for **probabilistic state estimation**. It maintains a belief, a probability distribution over the hidden state, and cycles through two steps at every time step. First, it predicts where the state probably moved (using a **motion model**). Then, it updates that prediction with the new **sensor measurement**.
- ❖ Performs recursive state estimation
- ❖ **Prediction** step to exploit the **controls**
- ❖ **Correction** step to exploit the **observations**



Kalman Filter Example

Prediction: where I think I am based on the speed

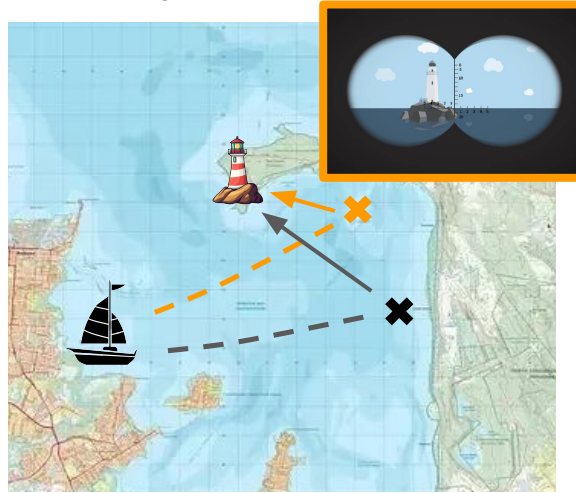


Kalman Filter Example

Prediction: where I think I am based on the speed

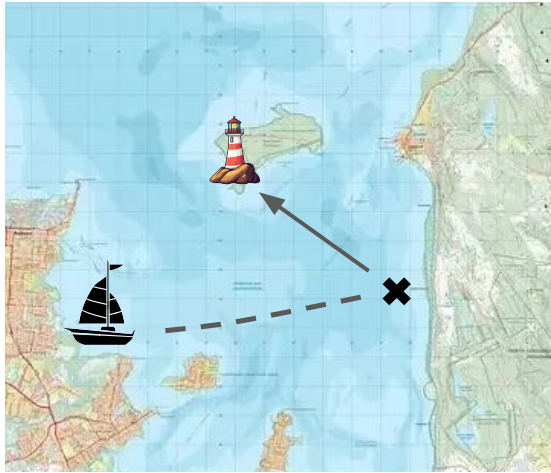


Observation: where I should be according to my observation

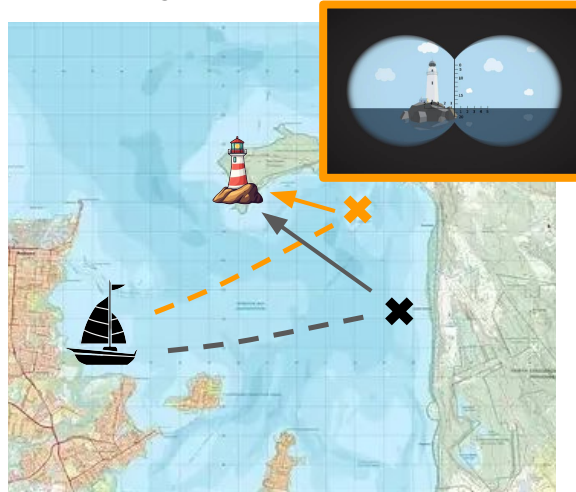


Kalman Filter Example

Prediction: where I think I am based on the speed



Observation: where I should be according to my observation



Correction: how do I fuse those two estimates?



State Estimation Problems

- ❖ Bayes Filter: tool for probabilistic state estimation
- ❖ **Prediction** step to exploit the **controls**

$$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1}) bel(x_{t-1}) dx_{t-1}$$

- ❖ **Correction** step to exploit the **observations**

$$bel(x_t) = \eta p(z_t \mid x_t) \overline{bel}(x_t)$$



Kalman Filter

- ❖ It is a Bayes Filter
- ❖ Performs recursive state estimation
- ❖ **Prediction** step to exploit the **controls**
- ❖ **Correction** step to exploit the **observations**

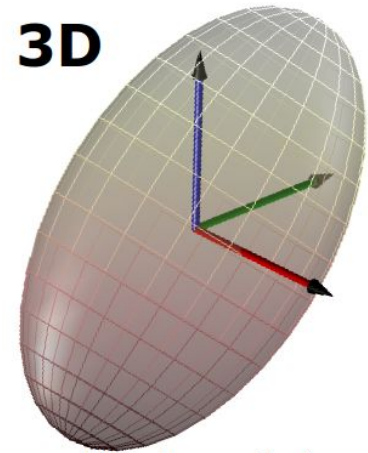
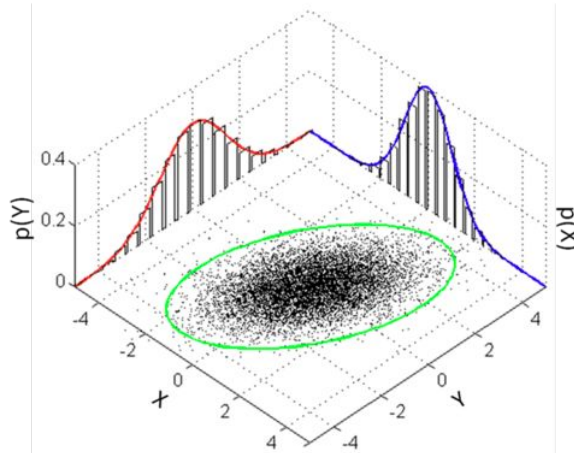
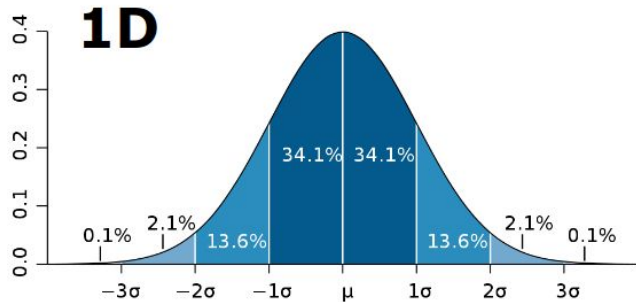
- ❖ **Optimal** solution for **linear** models and **Gaussian** distributions
- ❖ Result equivalent to least squares solution in a linear Gaussian world



Multivariate Gaussian Distribution

- Probability density function

$$p(\mathbf{x}) = \det(2\pi\mathbf{\Sigma})^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$



Properties: Marginalization and Conditioning

- **Given**

$$x = \begin{pmatrix} x_a \\ x_b \end{pmatrix} \quad p(x) = \mathcal{N}$$

- The **marginals** are **Gaussians**

$$p(x_a) = \mathcal{N} \quad p(x_b) = \mathcal{N}$$

- as well as the **conditionals**

$$p(x_a \mid x_b) = \mathcal{N} \quad p(x_b \mid x_a) = \mathcal{N}$$



Marginalization

- Given $p(x) = p(x_a, x_b) = \mathcal{N}(\mu, \Sigma)$

with
$$\mu = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix} \quad \Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}$$

- The **marginal distribution** is

with
$$p(x_a) = \int p(x_a, x_b) dx_b = \mathcal{N}(\mu, \Sigma)$$

$$\mu = \mu_a \quad \Sigma = \Sigma_{aa}$$



Conditioning

- Given $p(x) = p(x_a, x_b) = \mathcal{N}(\mu, \Sigma)$

with
$$\mu = \begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix} \quad \Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}$$

- The **conditional distribution** is

$$p(x_a | x_b) = \frac{p(x_a, x_b)}{p(x_b)} = \mathcal{N}(\mu, \Sigma)$$

with

$$\mu = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(x_b - \mu_b)$$

$$\Sigma = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$$



Marginalization and Conditioning

$$p\left(\begin{pmatrix} x_a \\ x_b \end{pmatrix}\right) = \mathcal{N}(\mu, \Sigma) = \mathcal{N}\left(\begin{pmatrix} \mu_a \\ \mu_b \end{pmatrix}, \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}\right)$$

Marginalization

$$p(x_a) = \mathcal{N}(\mu, \Sigma)$$

$$\mu = \mu_a$$

$$\Sigma = \Sigma_{aa}$$

Conditioning

$$p(x_a \mid x_b) = \mathcal{N}(\mu, \Sigma)$$

$$\mu = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b)$$

$$\Sigma = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$$



Linear Model

- Both models can be expressed through a linear function

$$f(x) = Ax + b$$

- A **Gaussian** that is transformed through a linear function **stays Gaussian**



Linear Model

- The Kalman filter assumes a **linear transition** and **observation model**
- Zero mean Gaussian noise

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$



Linear Model

- The Kalman filter assumes a **linear transition** and **observation model**
- Zero mean Gaussian noise

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$



Linear Model

- The Kalman filter assumes a **linear transition** and **observation model**
- Zero mean Gaussian noise

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$

A_t Matrix ($n \times n$) that describes how the state evolves from $t - 1$ to t without controls or noise.

B_t Matrix ($n \times l$) that describes how the control u_t changes the state from $t - 1$ to t .

C_t Matrix ($k \times n$) that describes how to map the state x_t to an observation z_t .

ϵ_t Random variables representing the process and measurement noise that are assumed to be independent and normally distributed with covariance R_t and Q_t respectively.



Linear Motion Model

- Multivariate Gaussian Distribution $p(\mathbf{x}) = \det(2\pi\mathbf{\Sigma})^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$
- Linear Model
$$\begin{aligned}x_t &= A_t x_{t-1} + B_t u_t + \epsilon_t \\z_t &= C_t x_t + \delta_t\end{aligned}$$

- Motion under Gaussian noise leads to

- R_t describes the noise of the motion



Linear Motion Model

- Multivariate Gaussian Distribution $p(\mathbf{x}) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$
- Linear Model
$$\begin{aligned}x_t &= A_t x_{t-1} + B_t u_t + \epsilon_t \\z_t &= C_t x_t + \delta_t\end{aligned}$$

- **Motion under Gaussian noise leads to**

$$\begin{aligned}p(x_t \mid u_t, x_{t-1}) &= \det(2\pi R_t)^{-\frac{1}{2}} \\&\exp\left(-\frac{1}{2}(x_t - A_t x_{t-1} - B_t u_t)^T R_t^{-1}(x_t - A_t x_{t-1} - B_t u_t)\right)\end{aligned}$$

- R_t describes the noise of the motion



Linear Motion Model

- Multivariate Gaussian Distribution $p(\mathbf{x}) = \det(2\pi\mathbf{\Sigma})^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$
- Linear Model
$$\begin{aligned}x_t &= A_t x_{t-1} + B_t u_t + \epsilon_t \\z_t &= C_t x_t + \delta_t\end{aligned}$$

- Measuring under Gaussian noise leads to

- Q_t describes the measurement noise



Linear Motion Model

- Multivariate Gaussian Distribution $p(\mathbf{x}) = \det(2\pi\mathbf{\Sigma})^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$
- Linear Model
$$\begin{aligned}x_t &= A_t x_{t-1} + B_t u_t + \epsilon_t \\z_t &= C_t x_t + \delta_t\end{aligned}$$

- **Measuring under Gaussian noise leads to**

$$p(z_t | x_t) = \det(2\pi Q_t)^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(z_t - C_t x_t)^T Q_t^{-1}(z_t - C_t x_t)\right)$$

- Q_t describes the measurement noise



Everything stays Gaussian

- ❖ Given an initial Gaussian belief, the belief stays Gaussian
- ❖ **Prediction** step to exploit the **controls**

$$\frac{\overline{bel}(x_t)}{????} = \int \frac{p(x_t \mid u_t, x_{t-1})}{????} \frac{bel(x_{t-1})}{????} dx_{t-1}$$

- ❖ **Correction** step to exploit the **observations**

$$\frac{bel(x_t)}{????} = \eta \frac{p(z_t \mid x_t)}{????} \frac{\overline{bel}(x_t)}{????}$$



Everything stays Gaussian

- ❖ Given an initial Gaussian belief, the belief stays Gaussian
- ❖ **Prediction** step to exploit the **controls**

$$\overline{bel}(x_t) = \int \underbrace{p(x_t \mid u_t, x_{t-1})}_{\text{Gaussian}} \underbrace{bel(x_{t-1})}_{\text{Gaussian}} dx_{t-1}$$

- ❖ **Correction** step to exploit the **observations**

$$bel(x_t) = \eta \underbrace{p(z_t \mid x_t)}_{\text{Gaussian}} \underbrace{\overline{bel}(x_t)}_{\text{????}}$$

The product of two
Gaussian is a again a
Gaussian



Everything stays Gaussian

❖ We can write $\overline{bel}(x_t)$

$$\begin{aligned} &= \int p(x_t \mid u_t, x_{t-1}) \text{bel}(x_{t-1}) dx_{t-1} \\ &= \eta \int \exp \left(-\frac{1}{2} (x_t - A_t x_{t-1} - B_t u_t)^T R_t^{-1} (x_t - A_t x_{t-1} - B_t u_t) \right) \\ &\quad \exp \left(-\frac{1}{2} (x_{t-1} - \mu_{t-1})^T \Sigma_{t-1}^{-1} (x_{t-1} - \mu_{t-1}) \right) dx_{t-1} \end{aligned}$$

❖ and thus

$$\begin{aligned} \overline{bel}(x_t) &= \eta \int \exp(-L_t) dx_{t-1} \\ L_t &= \frac{1}{2} (x_t - A_t x_{t-1} - B_t u_t)^T R_t^{-1} (x_t - A_t x_{t-1} - B_t u_t) \\ &\quad + \frac{1}{2} (x_{t-1} - \mu_{t-1})^T \Sigma_{t-1}^{-1} (x_{t-1} - \mu_{t-1}) \end{aligned}$$



Everything stays Gaussian

- ❖ We can split L_t in a part that depends on x_t and on x_t, x_{t-1}

$$L_t = L_t(x_{t-1}, x_t) + L_t(x_t)$$

- ❖ Thus

$$\begin{aligned}\overline{bel}(x_t) &= \eta \int \exp(-L_t(x_{t-1}, x_t) - L_t(x_t)) dx_{t-1} \\ &= \underbrace{\eta \exp(-L_t(x_t))}_{\text{Gaussian}} \underbrace{\int \exp(-L_t(x_{t-1}, x_t)) dx_{t-1}}_{\text{Marginalization}}\end{aligned}$$

Details: Probabilistic Robotics, Ch. 3.2 (p. 46-49)



Everything stays Gaussian

- ❖ Given an initial Gaussian belief, the belief stays Gaussian
- ❖ **Prediction** step to exploit the **controls**

$$\underbrace{\overline{bel}(x_t)}_{\text{Gaussian}} = \underbrace{\int \underbrace{p(x_t \mid u_t, x_{t-1})}_{\text{Gaussian}} \underbrace{bel(x_{t-1})}_{\text{Gaussian}} dx_{t-1}}_{\text{Gaussian}}$$

- ❖ **Correction** step to exploit the **observations**

$$\underbrace{bel(x_t)}_{\text{Gaussian}} = \underbrace{\eta}_{\text{Gaussian}} \underbrace{p(z_t \mid x_t)}_{\text{Gaussian}} \underbrace{\overline{bel}(x_t)}_{\text{Gaussian}}$$

**Everything stays
Gaussian!**



To Derive the Kalman Filter Algorithm, One Exploits

- ❖ Product of two Gaussians is a Gaussian
- ❖ Gaussians stays Gaussians under linear transformations
- ❖ Marginal and conditional distribution of a Gaussian stays a Gaussian
- ❖ Computing mean and covariance of the marginal and conditional of a Gaussian
- ❖ Matrix inversion lemma
- ❖ ...

This leads us to ...



Kalman Filter Algorithm

1: **Kalman_filter** ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$) :

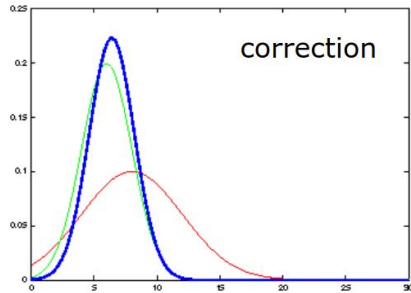
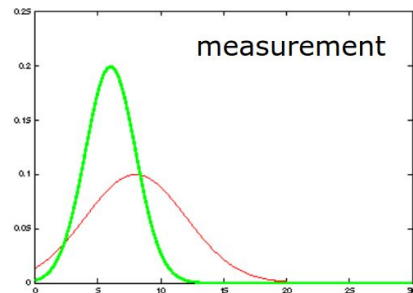
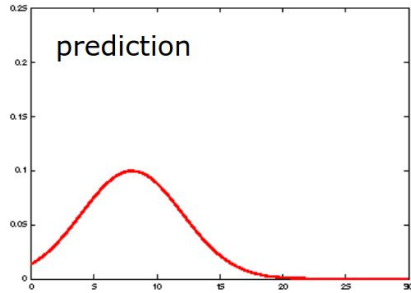
$$\left. \begin{array}{l} 2: \quad \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ 3: \quad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \end{array} \right\} \text{Prediction}$$

$$\left. \begin{array}{l} 4: \quad K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \\ 5: \quad \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ 6: \quad \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{array} \right\} \text{Correction}$$

7: *return* μ_t, Σ_t



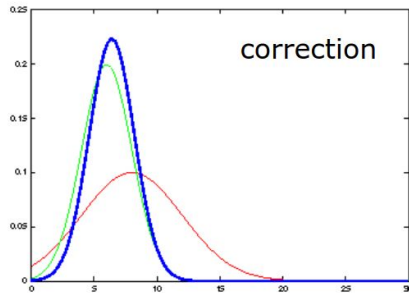
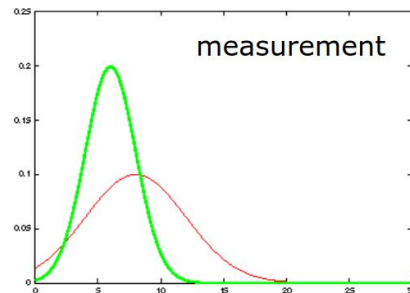
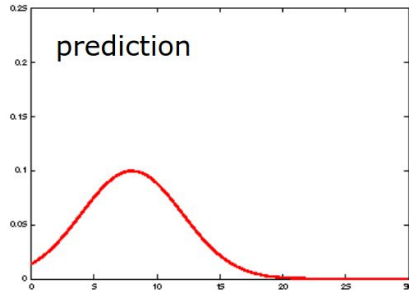
1D Kalman Filter Example



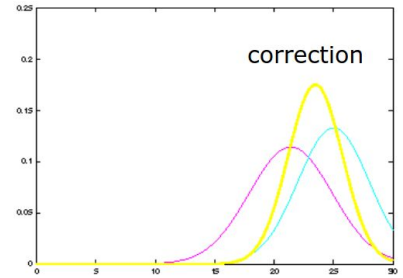
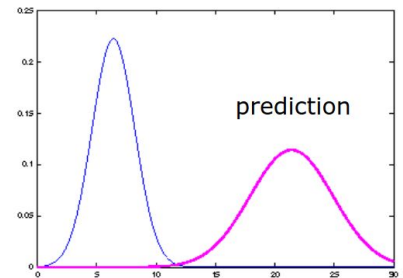
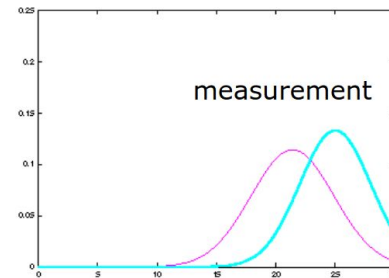
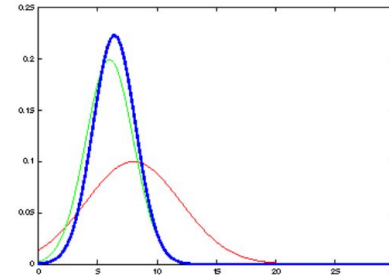
It's a weighted mean!



1D Kalman Filter Example



It's a weighted mean!



Exercise: Perfect Sensor $Q_t = 0$

1: **Kalman_filter** ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\left. \begin{array}{l} 2: \quad \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ 3: \quad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \end{array} \right\} \text{Prediction}$$

$$\left. \begin{array}{l} 4: \quad K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \\ 5: \quad \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ 6: \quad \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{array} \right\} \text{Correction}$$

7: *return* μ_t, Σ_t



Exercise: Perfect Sensor $Q_t = 0$

1: **Kalman_filter** ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\left. \begin{array}{l} 2: \quad \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ 3: \quad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \end{array} \right\} \text{Prediction}$$

$$\left. \begin{array}{l} 4: \quad K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \\ 5: \quad \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ 6: \quad \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{array} \right\} \text{Correction}$$

7: *return* μ_t, Σ_t

$$\begin{aligned} K_t &= \bar{\Sigma}_t C_t^T C_t^{-T} \bar{\Sigma}_t^{-1} C_t^{-1} = C_t^{-1} \\ \mu_t &= \bar{\mu}_t + C_t^{-1} (z_t - C_t \bar{\mu}_t) = \\ &\quad \bar{\mu}_t + C_t^{-1} z_t - C_t^{-1} C_t \bar{\mu}_t = C_t^{-1} z_t \\ \Sigma_t &= (I - C_t^{-1} C_t) \bar{\Sigma}_t = 0 \end{aligned}$$



Exercise: Broken Sensor $Q_t = \infty$

1: **Kalman_filter** ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$) :

$$\left. \begin{array}{l} 2: \quad \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ 3: \quad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \end{array} \right\} \text{Prediction}$$

$$\left. \begin{array}{l} 4: \quad K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \\ 5: \quad \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ 6: \quad \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{array} \right\} \text{Correction}$$

7: *return* μ_t, Σ_t



Exercise: Broken Sensor $Q_t = \infty$

1: **Kalman_filter** ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):

$$\left. \begin{array}{l} 2: \quad \bar{\mu}_t = A_t \mu_{t-1} + B_t u_t \\ 3: \quad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t \end{array} \right\} \text{Prediction}$$

$$\left. \begin{array}{l} 4: \quad K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1} \\ 5: \quad \mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t) \\ 6: \quad \Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \end{array} \right\} \text{Correction}$$

7: *return* μ_t, Σ_t

$$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + \infty)^{-1} = 0$$

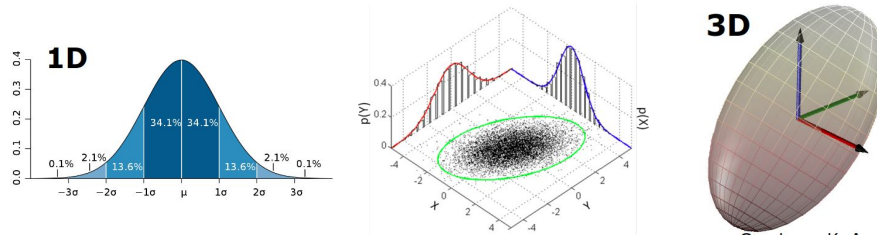
$$\underline{\mu_t = \bar{\mu}_t}$$

$$\underline{\Sigma_t = \bar{\Sigma}_t}$$



Kalman Filter Assumptions

- Gaussian distributions and noise



- Linear motion and observation model

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$

- What if this is not the case?



Non-linear Dynamic Systems

- Most realistic problems (in robotics) involve nonlinear functions

$$\cancel{x_t = A_t x_{t-1} + B_t u_t + \epsilon_t}$$



$$x_t = g(u_t, x_{t-1}) + \epsilon_t$$

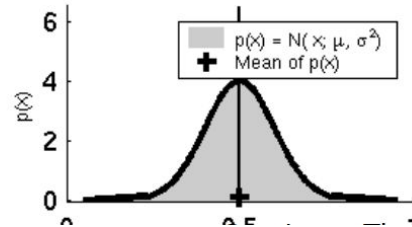
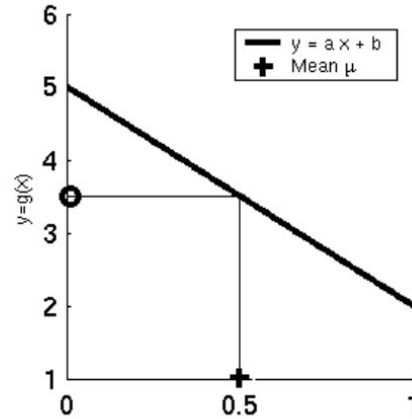
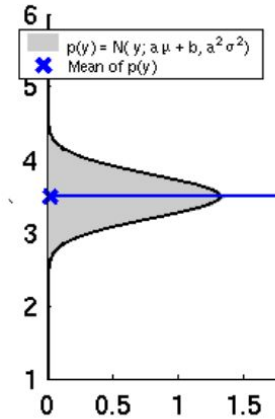
$$\cancel{z_t = C_t x_t + \delta_t}$$



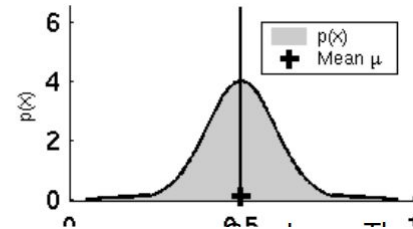
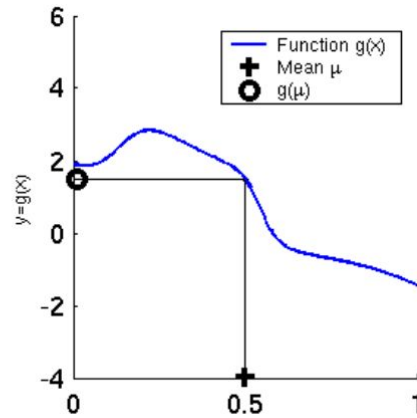
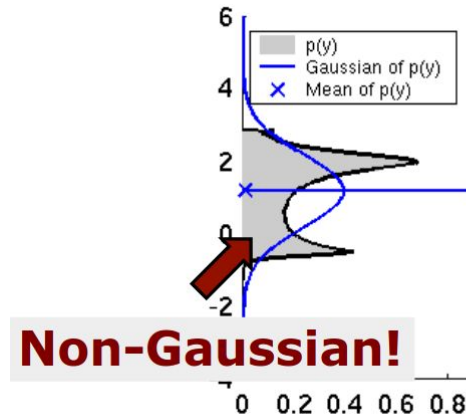
$$z_t = h(x_t) + \delta_t$$



Linearity Assumption Revisited



Non-Linear Function



Non-Gaussian Distributions

- ❖ The non-linear functions lead to non-Gaussian distributions
- ❖ Kalman filter is not applicable anymore!

What can be done to resolve this?

Local Linearization



Non-Gaussian Distributions

- Prediction:

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \underbrace{\frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}}_{=: G_t} (x_{t-1} - \mu_{t-1})$$

- Correction:

$$h(x_t) \approx h(\bar{\mu}_t) + \underbrace{\frac{\partial h(\bar{\mu}_t)}{\partial x_t}}_{=: H_t} (x_t - \bar{\mu}_t)$$

Jacobian matrices



Reminder: Jacobian Matrix

- ❖ It is a non-square matrix $m \times n$ in general
- ❖ Given a vector-valued function

$$g(x) = \begin{pmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_m(x) \end{pmatrix}$$

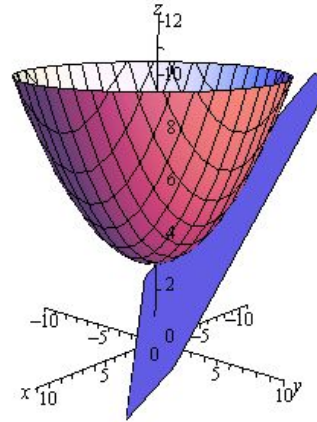
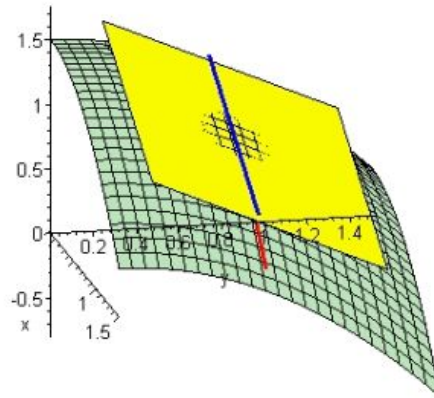
- ❖ The Jacobian matrix is defined as

$$G_x = \begin{pmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} & \cdots & \frac{\partial g_1}{\partial x_n} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} & \cdots & \frac{\partial g_2}{\partial x_n} \\ \vdots & \vdots & \cdots & \vdots \\ \frac{\partial g_m}{\partial x_1} & \frac{\partial g_m}{\partial x_2} & \cdots & \frac{\partial g_m}{\partial x_n} \end{pmatrix}$$



Reminder: Jacobian Matrix

- ❖ It is the orientation of the tangent plane to the vector-valued function at a given point



- ❖ Generalizes the gradient of a scalar valued function



EKF Linearization: First Order Taylor Expansion

- Prediction:

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \underbrace{\frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}}_{=: G_t} (x_{t-1} - \mu_{t-1})$$

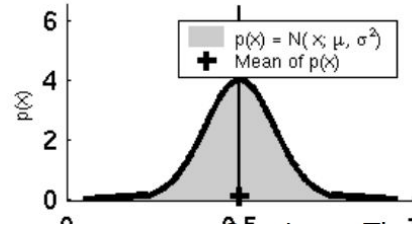
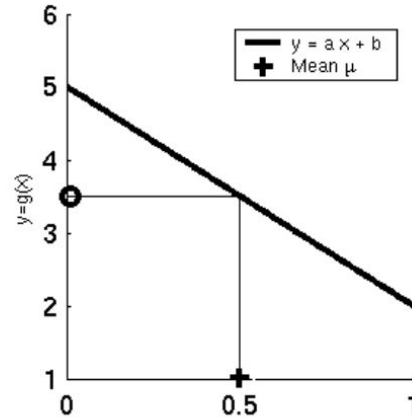
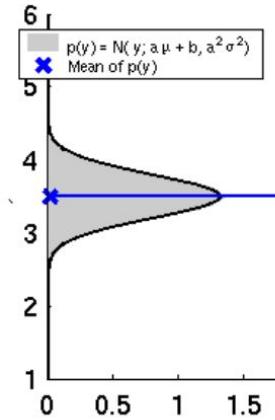
- Correction:

$$h(x_t) \approx h(\bar{\mu}_t) + \underbrace{\frac{\partial h(\bar{\mu}_t)}{\partial x_t}}_{=: H_t} (x_t - \bar{\mu}_t)$$

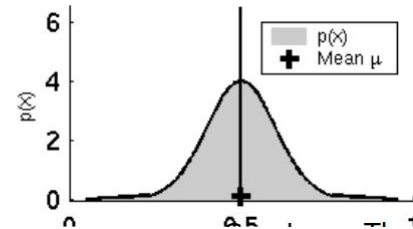
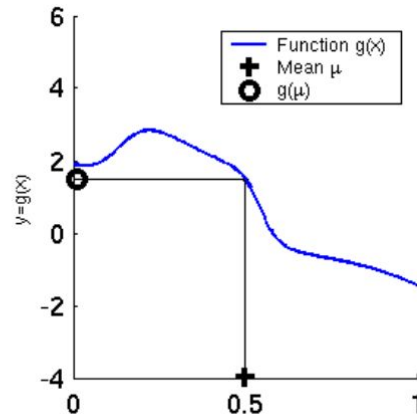
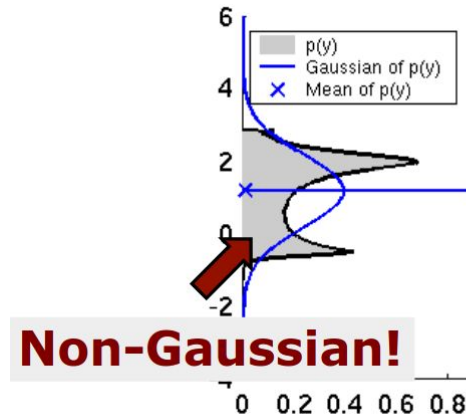
Linear functions!



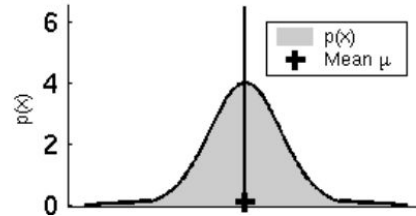
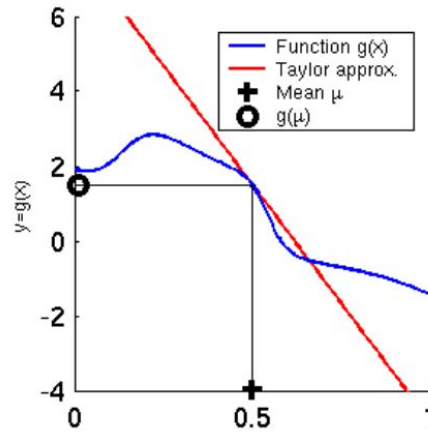
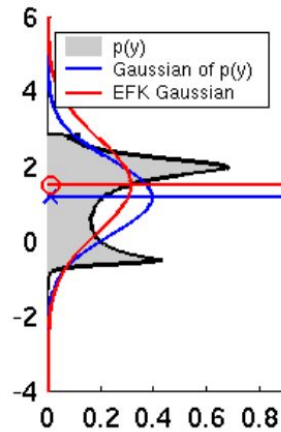
EKF Linearization



EKF Linearization



EKF Linearization



Linearized Motion Model

- The linearized model leads to

$$p(x_t \mid u_t, x_{t-1}) \approx \det(2\pi R_t)^{-\frac{1}{2}} \exp \left(-\frac{1}{2} (x_t - g(u_t, \mu_{t-1}) - G_t (x_{t-1} - \mu_{t-1}))^T R_t^{-1} \underbrace{(x_t - g(u_t, \mu_{t-1}) - G_t (x_{t-1} - \mu_{t-1}))}_{\text{linearized model}} \right)$$

- R_t describes the noise of the motion



Linearized Observation Model

- The linearized model leads to

$$p(z_t | x_t) = \det(2\pi Q_t)^{-\frac{1}{2}} \exp \left(-\frac{1}{2} (z_t - h(\bar{\mu}_t) - H_t (x_t - \bar{\mu}_t))^T Q_t^{-1} (z_t - \underbrace{h(\bar{\mu}_t) - H_t (x_t - \bar{\mu}_t)}_{\text{linearized model}}) \right)$$

- Q_t describes the noise of the motion



Extended Kalman Filter Algorithm

1:	KF $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$		EKF $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
2:	$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$	Prediction	$\bar{\mu}_t = g(u_t, \mu_{t-1})$
3:	$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$		$\bar{\Sigma}_t = \underline{G}_t \Sigma_{t-1} \underline{G}_t^T + R_t$
4:	$K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$	Correction	$K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
5:	$\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$		$\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
6:	$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$		$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
7:	return μ_t, Σ_t		return μ_t, Σ_t



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
 - 2: $\bar{\mu}_t = g(u_t, \mu_{t-1})$
 - 3: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$
 - 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
 - 5: $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
 - 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
 - 7: *return* μ_t, Σ_t
- } Prediction
- } Correction



Extended Kalman Filter Summary

- ❖ Extension of the Kalman Filter
- ❖ One way to handle the non-linearities
- ❖ Performs local linearizations
- ❖ Works well in practice for moderate non-linearities
- ❖ Large uncertainty leads to increased approximation errors



EKF SLAM

Given

- The sent controls commands

$$u_{1:T} = \{u_1, u_2, u_3, \dots, u_T\}$$

- Observations

$$z_{1:T} = \{z_1, z_2, z_3, \dots, z_T\}$$

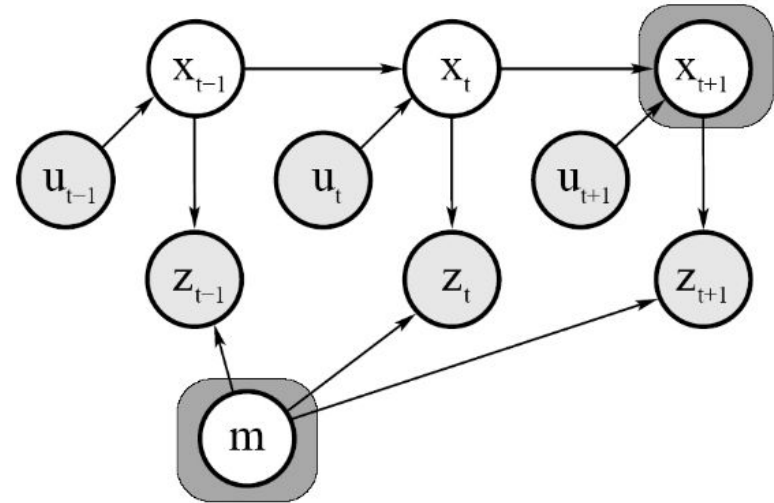
Wanted

- Map of the environment

m

- Path (or current pose) of the vehicle

$$x_{0:T} = \{x_0, x_1, x_2, \dots, x_T\}$$



State Representation

$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$

- More compactly

$$\underbrace{\begin{pmatrix} x \\ m \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{pmatrix}}_{\Sigma}$$



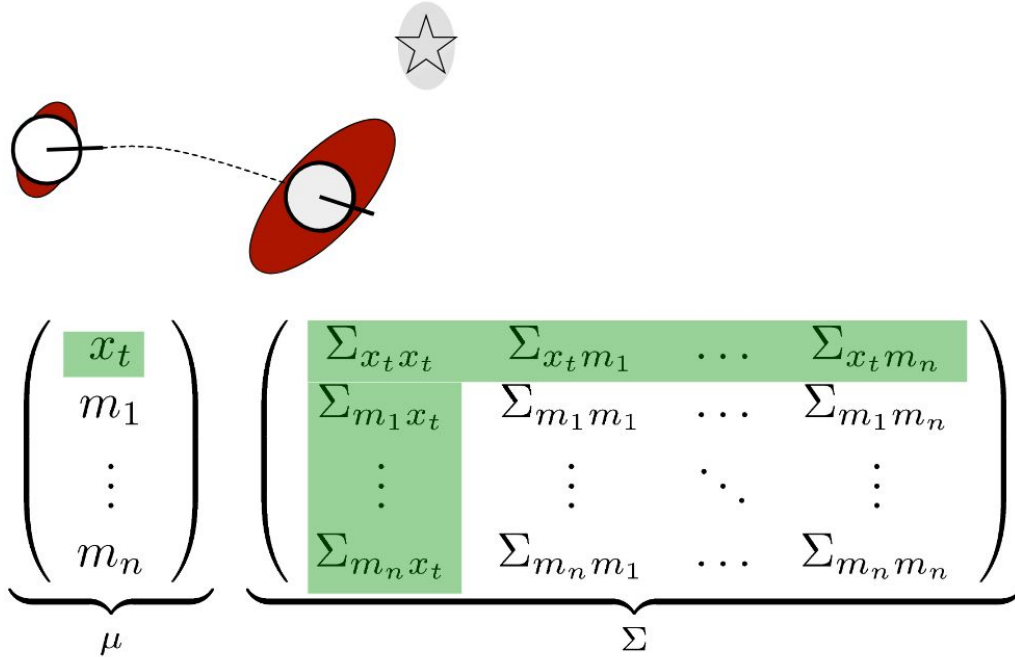
EKF SLAM: Initial State



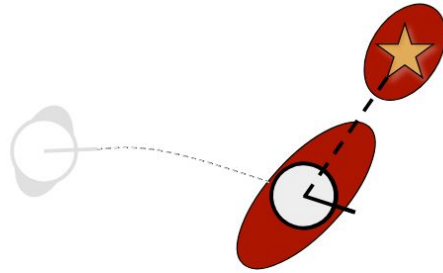
$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$



EKF SLAM: Predicted Motion



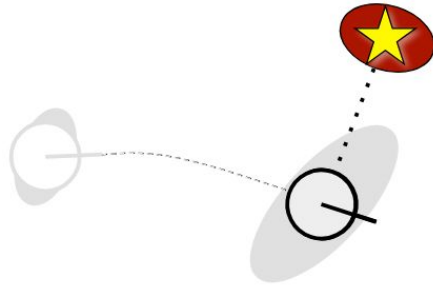
EKF SLAM: Predicted Measurement



$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$



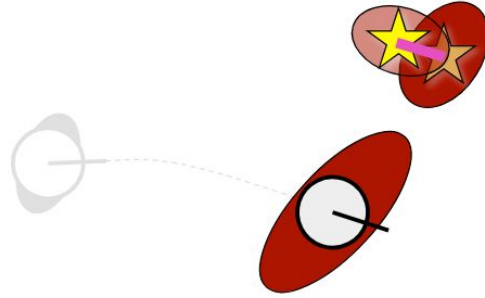
EKF SLAM: Measurement



$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$



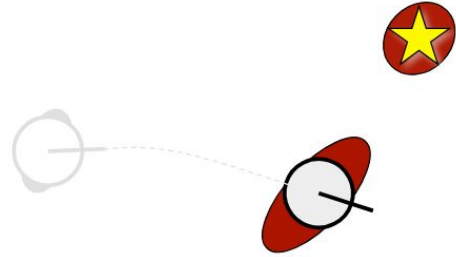
EKF SLAM: Data Association and Difference Between $h(x)$ and z



$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$



EKF SLAM: Update State



$$\underbrace{\begin{pmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{pmatrix}}_{\mu} \quad \underbrace{\begin{pmatrix} \Sigma_{x_t x_t} & \Sigma_{x_t m_1} & \cdots & \Sigma_{x_t m_n} \\ \Sigma_{m_1 x_t} & \Sigma_{m_1 m_1} & \cdots & \Sigma_{m_1 m_n} \\ \vdots & \vdots & \ddots & \vdots \\ \Sigma_{m_n x_t} & \Sigma_{m_n m_1} & \cdots & \Sigma_{m_n m_n} \end{pmatrix}}_{\Sigma}$$



EKF SLAM Application

- ❖ Application of the EKF to SLAM
- ❖ Estimate robot's **pose** and locations of **landmarks** in the environment
- ❖ Assumption: known correspondences
- ❖ State space (for the 2D plane) is:

$$x_t = \left(\underbrace{x, y, \theta}_{\text{robot's pose}}, \underbrace{m_{1,x}, m_{1,y}}_{\text{landmark 1}}, \dots, \underbrace{m_{n,x}, m_{n,y}}_{\text{landmark n}} \right)^T$$



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
 - 2: $\bar{\mu}_t = g(u_t, \mu_{t-1})$
 - 3: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$
 - 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
 - 5: $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
 - 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
 - 7: *return* μ_t, Σ_t
- } Prediction
- } Correction



Prediction Step (Motion)

- Goal: Update state space based on the motion
- Motion in the plane

$$\begin{pmatrix} x' \\ y' \\ \theta' \end{pmatrix} = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}$$

$g_{x,y,\theta}(u_t, (x, y, \theta)^T)$

- How to map that to the $2N + 3$ dim state used in the EKF?

$$\begin{pmatrix} x' \\ y' \\ \theta' \\ \vdots \end{pmatrix} = \begin{pmatrix} x \\ y \\ \theta \\ \vdots \end{pmatrix} + \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \dots 0 \\ 0 & 1 & 0 & 0 \dots 0 \\ 0 & 0 & 1 & 0 \dots 0 \end{pmatrix}^T}_{F_x^T} \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}$$

$g(u_t, x_t)$



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
 - 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~
 - 3: $\bar{\Sigma}_t = \boxed{G_t} \Sigma_{t-1} G_t^T + R_t$
 - 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
 - 5: $\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$
 - 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
 - 7: *return* μ_t, Σ_t
- } Prediction
- } Correction



Update Covariance

- The function g only affect the motion and not the landmarks

Jacobian of the motion (3x3)

$$G_t = \begin{pmatrix} G_t^x & 0 \\ 0 & I \end{pmatrix}$$

Identity (2N x 2N)



Jacobian of the motion

$$G_t^x = \frac{\partial}{\partial (x, y, \theta)^T} \left[\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \right]$$



Jacobian of the motion

$$\begin{aligned} G_t^x &= \frac{\partial}{\partial(x, y, \theta)^T} \left[\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \right] \\ &= I + \frac{\partial}{\partial(x, y, \theta)^T} \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \end{aligned}$$



Jacobian of the motion

$$\begin{aligned} G_t^x &= \frac{\partial}{\partial(x, y, \theta)^T} \left[\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \right] \\ &= I + \frac{\partial}{\partial(x, y, \theta)^T} \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \\ &= I + \begin{pmatrix} 0 & 0 & -\frac{v_t}{\omega_t} \cos \theta + \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ 0 & 0 & -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ 0 & 0 & 0 \end{pmatrix} \end{aligned}$$



Jacobian of the motion

$$\begin{aligned} G_t^x &= \frac{\partial}{\partial(x, y, \theta)^T} \left[\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} + \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \right] \\ &= I + \frac{\partial}{\partial(x, y, \theta)^T} \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \theta - \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix} \\ &= I + \begin{pmatrix} 0 & 0 & -\frac{v_t}{\omega_t} \cos \theta + \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ 0 & 0 & -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ 0 & 0 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 & -\frac{v_t}{\omega_t} \cos \theta + \frac{v_t}{\omega_t} \cos(\theta + \omega_t \Delta t) \\ 0 & 1 & -\frac{v_t}{\omega_t} \sin \theta + \frac{v_t}{\omega_t} \sin(\theta + \omega_t \Delta t) \\ 0 & 0 & 1 \end{pmatrix} \end{aligned}$$



Extended Kalman Filter Algorithm

- 1: **EKF** ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$)
 - 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~
 - 3: $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$
- } Prediction


$$\begin{aligned}\bar{\Sigma}_t &= G_t \Sigma_{t-1} G_t^T + R_t \\ &= \begin{pmatrix} G_t^x & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} \Sigma_{xx} & \Sigma_{xm} \\ \Sigma_{mx} & \Sigma_{mm} \end{pmatrix} \begin{pmatrix} (G_t^x)^T & 0 \\ 0 & I \end{pmatrix} + R_t \\ &= \begin{pmatrix} G_t^x \Sigma_{xx} (G_t^x)^T & G_t^x \Sigma_{xm} \\ (G_t^x \Sigma_{xm})^T & \Sigma_{mm} \end{pmatrix} + R_t\end{aligned}$$



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ } Prediction
- 4: $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$ } Correction
- 7: *return* μ_t, Σ_t



EKF SLAM: Prediction Step

EKF_SLAM_Prediction($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t, c_t, R_t$):

$$2: \quad F_x = \begin{pmatrix} 1 & 0 & 0 & 0 \dots 0 \\ 0 & 1 & 0 & 0 \dots 0 \\ 0 & 0 & 1 & 0 \dots 0 \end{pmatrix}$$

$$3: \quad \bar{\mu}_t = \mu_{t-1} + F_x^T \begin{pmatrix} -\frac{v_t}{\omega_t} \sin \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \sin(\mu_{t-1, \theta} + \omega_t \Delta t) \\ \frac{v_t}{\omega_t} \cos \mu_{t-1, \theta} - \frac{v_t}{\omega_t} \cos(\mu_{t-1, \theta} + \omega_t \Delta t) \\ \omega_t \Delta t \end{pmatrix}$$

$$4: \quad G_t = I + F_x^T \begin{pmatrix} 0 & 0 & -\frac{v_t}{\omega_t} \cos \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \cos(\mu_{t-1, \theta} + \omega_t \Delta t) \\ 0 & 0 & -\frac{v_t}{\omega_t} \sin \mu_{t-1, \theta} + \frac{v_t}{\omega_t} \sin(\mu_{t-1, \theta} + \omega_t \Delta t) \\ 0 & 0 & 0 \end{pmatrix} F_x$$

$$5: \quad \bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + \underbrace{F_x^T R_t^x F_x}_{R_t}$$



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ } Prediction
- 4: $K_t = \bar{\Sigma}_t \boxed{H_t^T} (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t (z_t - \boxed{h(\bar{\mu}_t)})$ } Correction
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7: *return* μ_t, Σ_t



Range-Bearing Observation

- Range-Bearing Observation $z_t^i = (r_t^i, \phi_t^i)^T$
- Compute expected observation according to the current estimate

$$\begin{aligned}\delta &= \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix} \\ q &= \delta^T \delta \\ \hat{z}_t^i &= \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix} \\ &= h(\bar{\mu}_t)\end{aligned}$$



Jacobian for the Observation

- Based on

$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$

$$q = \delta^T \delta$$

$$\begin{aligned} \hat{z}_t^i &= \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix} \\ &= h(\bar{\mu}_t) \end{aligned}$$

- Compute the Jacobian

$$\text{low } H_t^i = \frac{\partial h(\bar{\mu}_t)}{\partial \bar{\mu}_t}$$



low-dim space $(x, y, \theta, m_{j,x}, m_{j,y})$



Jacobian for the Observation

- Based on

$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$

$$q = \delta^T \delta$$

$$\begin{aligned} \hat{z}_t^i &= \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix} \\ &= h(\bar{\mu}_t) \end{aligned}$$

- Compute the Jacobian

$$\begin{aligned} {}^{\text{low}}H_t^i &= \frac{\partial h(\bar{\mu}_t)}{\partial \bar{\mu}_t} \\ &= \begin{pmatrix} \frac{\partial \sqrt{q}}{\partial x} & \frac{\partial \sqrt{q}}{\partial y} & \dots \\ \frac{\partial \text{atan2}(\dots)}{\partial x} & \frac{\partial \text{atan2}(\dots)}{\partial y} & \dots \end{pmatrix} \end{aligned}$$



Jacobian for the Observation

- Based on

$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$

$$q = \delta^T \delta$$

$$\begin{aligned} \hat{z}_t^i &= \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix} \\ &= h(\bar{\mu}_t) \end{aligned}$$

- We obtain (by applying the chain rule)

$$\begin{aligned} \frac{\partial \sqrt{q}}{\partial x} &= \frac{1}{2} \frac{1}{\sqrt{q}} 2 \delta_x (-1) \\ &= \frac{1}{q} (-\sqrt{q} \delta_x) \end{aligned}$$



Jacobian for the Observation

- Based on

$$\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$$

$$q = \delta^T \delta$$

$$\begin{aligned} \hat{z}_t^i &= \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix} \\ &= h(\bar{\mu}_t) \end{aligned}$$

- Compute the Jacobian

$$\begin{aligned} {}^{\text{low}}H_t^i &= \frac{\partial h(\bar{\mu}_t)}{\partial \bar{\mu}_t} \\ &= \frac{1}{q} \begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & +\sqrt{q}\delta_x & \sqrt{q}\delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x \end{pmatrix} \end{aligned}$$



Jacobian for the Observation

- Use the computed Jacobian

$$\begin{aligned} {}^{\text{low}}H_t^i &= \frac{\partial h(\bar{\mu}_t)}{\partial \bar{\mu}_t} \\ &= \frac{1}{q} \begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & +\sqrt{q}\delta_x & \sqrt{q}\delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & \delta_x \end{pmatrix} \end{aligned}$$

- map it to the high dimensional space

$$H_t^i = {}^{\text{low}}H_t^i F_{x,j}$$

$\downarrow$
 $F_{x,j} =$

$\begin{pmatrix} 1 & 0 & 0 & 0 \dots 0 & 0 & 0 & 0 \dots 0 \\ 0 & 1 & 0 & 0 \dots 0 & 0 & 0 & 0 \dots 0 \\ 0 & 0 & 1 & 0 \dots 0 & 0 & 0 & 0 \dots 0 \\ 0 & 0 & 0 & 0 \dots 0 & 1 & 0 & 0 \dots 0 \\ 0 & 0 & 0 & \underbrace{0 \dots 0}_{2j-2} & 0 & 1 & \underbrace{0 \dots 0}_{2N-2j} \end{pmatrix}$



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ } Prediction
- 4: $K_t = \bar{\Sigma}_t \boxed{H_t^T} (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5: $\mu_t = \bar{\mu}_t + K_t (z_t - \boxed{h(\bar{\mu}_t)})$ } Correction
- 6: $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7: *return* μ_t, Σ_t



Extended Kalman Filter Algorithm

- 1: **EKF** $(\mu_{t-1}, \Sigma_{t-1}, u_t, z_t)$
- 2: ~~$\bar{\mu}_t = g(u_t, \mu_{t-1})$~~
- 3: ~~$\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$~~ } Prediction
- 4: ~~$K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$~~
- 5: ~~$\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$~~ } Correction
- 6: ~~$\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$~~
- 7: *return* μ_t, Σ_t



EKF SLAM Correction

EKF_SLAM_Correction

```
6:   $Q_t = \begin{pmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{pmatrix}$ 
7:  for all observed features  $z_t^i = (r_t^i, \phi_t^i)^T$  do
8:     $j = c_t^i$ 
9:    if landmark  $j$  never seen before
10:       $\begin{pmatrix} \bar{\mu}_{j,x} \\ \bar{\mu}_{j,y} \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{t,x} \\ \bar{\mu}_{t,y} \end{pmatrix} + \begin{pmatrix} r_t^i \cos(\phi_t^i + \bar{\mu}_{t,\theta}) \\ r_t^i \sin(\phi_t^i + \bar{\mu}_{t,\theta}) \end{pmatrix}$ 
11:    endif
12:     $\delta = \begin{pmatrix} \delta_x \\ \delta_y \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{j,x} - \bar{\mu}_{t,x} \\ \bar{\mu}_{j,y} - \bar{\mu}_{t,y} \end{pmatrix}$ 
13:     $q = \delta^T \delta$ 
14:     $\hat{z}_t^i = \begin{pmatrix} \sqrt{q} \\ \text{atan2}(\delta_y, \delta_x) - \bar{\mu}_{t,\theta} \end{pmatrix}$ 
```



EKF SLAM Correction

$$\begin{aligned}
 15: \quad F_{x,j} &= \begin{pmatrix} 1 & 0 & 0 & 0 \cdots 0 & 0 & 0 & 0 \cdots 0 \\ 0 & 1 & 0 & 0 \cdots 0 & 0 & 0 & 0 \cdots 0 \\ 0 & 0 & 1 & 0 \cdots 0 & 0 & 0 & 0 \cdots 0 \\ 0 & 0 & 0 & 0 \cdots 0 & 1 & 0 & 0 \cdots 0 \\ 0 & 0 & 0 & \underbrace{0 \cdots 0}_{2j-2} & 0 & 1 & \underbrace{0 \cdots 0}_{2N-2j} \end{pmatrix} \\
 16: \quad H_t^i &= \frac{1}{q} \begin{pmatrix} -\sqrt{q}\delta_x & -\sqrt{q}\delta_y & 0 & +\sqrt{q}\delta_x & \sqrt{q}\delta_y \\ \delta_y & -\delta_x & -q & -\delta_y & +\delta_x \end{pmatrix} F_{x,j} \\
 17: \quad K_t^i &= \bar{\Sigma}_t H_t^{iT} (H_t^i \bar{\Sigma}_t H_t^{iT} + Q_t)^{-1} \\
 18: \quad \bar{\mu}_t &= \bar{\mu}_t + K_t^i (z_t^i - \hat{z}_t^i) \\
 19: \quad \bar{\Sigma}_t &= (I - K_t^i H_t^i) \bar{\Sigma}_t \\
 20: \quad & \text{endfor} \\
 21: \quad \mu_t &= \bar{\mu}_t \\
 22: \quad \Sigma_t &= \bar{\Sigma}_t \\
 23: \quad & \text{return } \mu_t, \Sigma_t
 \end{aligned}$$



Initialization

- Platform starts in its own reference frame (all landmarks unknown)
- $2N + 3$ dimensions

$$\mu_0 = (0 \ 0 \ 0 \ \dots \ 0)^T$$

$$\Sigma_0 = \begin{pmatrix} 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & \infty & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \infty \end{pmatrix}$$

- If landmark has not been observed, we can initialize it with:

$$\begin{pmatrix} \bar{\mu}_{j,x} \\ \bar{\mu}_{j,y} \end{pmatrix} = \begin{pmatrix} \bar{\mu}_{t,x} \\ \bar{\mu}_{t,y} \end{pmatrix} + \begin{pmatrix} r_t^i \cos(\phi_t^i + \bar{\mu}_{t,\theta}) \\ r_t^i \sin(\phi_t^i + \bar{\mu}_{t,\theta}) \end{pmatrix}$$

location of
landmark j

estimated
location of
the platform

relative
measurement



EKF -SLAM

